



Indian Institute of Technology Bombay

EE677– Introduction to VLSI CAD, Autumn 2025–26

Report on

Assignment 1: Combinational circuit simulator

<i>Author:</i>	Hardik Khariwal(22B3954)
<i>Course Instructor:</i>	Prof. Virendra Singh
<i>Date:</i>	28 August 2025

Abstract

This report presents the design and implementation of a VHDL gate-level logic simulator using Python. The tool automates the flow of parsing VHDL netlists, converting them into JSON-based intermediate representation, generating simplified Boolean logic expressions, and simulating these expressions with input vectors to produce outputs. This project provides foundational insights into digital circuit simulation and serves as an introductory CAD flow for VLSI design.

Contents

1	Introduction	4
2	Objectives	4
3	Methodology	4
3.1	Input Files	4
3.2	Intermediate Files	4
3.3	Output Files	5
4	Implementation	5
4.1	Project Structure	5
4.2	Workflow	6
4.3	Example	6
5	Results	7
6	Conclusion	7

List of Figures

1	Graph representation of circuit generated from JSON	5
---	---	---

1 Introduction

The purpose of this assignment is to design and implement a VHDL gate-level logic simulator using Python.

The simulator automates the following tasks:

- Parsing a VHDL gate-level netlist.
- Converting the netlist into an intermediate JSON representation.
- Generating simplified logic expressions (SLC).
- Evaluating these expressions using a set of simulation vectors.

This work provides a simplified yet powerful tool for understanding digital circuit simulation, which is fundamental in VLSI CAD flows.

2 Objectives

The main objectives of the assignment are:

- Parse VHDL netlists and extract gate-level connectivity.
- Represent the circuit as an intermediate JSON file.
- Generate Simplified Logic Expressions (SLC) from JSON.
- Evaluate input test vectors and produce output simulation results.
- Store intermediate files for debugging and transparency.
- Provide modular code organization for professional usage.

3 Methodology

3.1 Input Files

- **Netlist File (.vhd1)**: Gate-level description of the circuit.
- **Simulation Vectors (.txt)**: Contains test cases for input signals.

3.2 Intermediate Files

- **JSON File (.json)**: Graph-based representation of the circuit.
- **SLC File (.txt)**: Simplified Boolean expression extracted from JSON.

3.3 Output Files

- **Simulation Output (.txt):** Contains evaluated outputs for all test vectors.

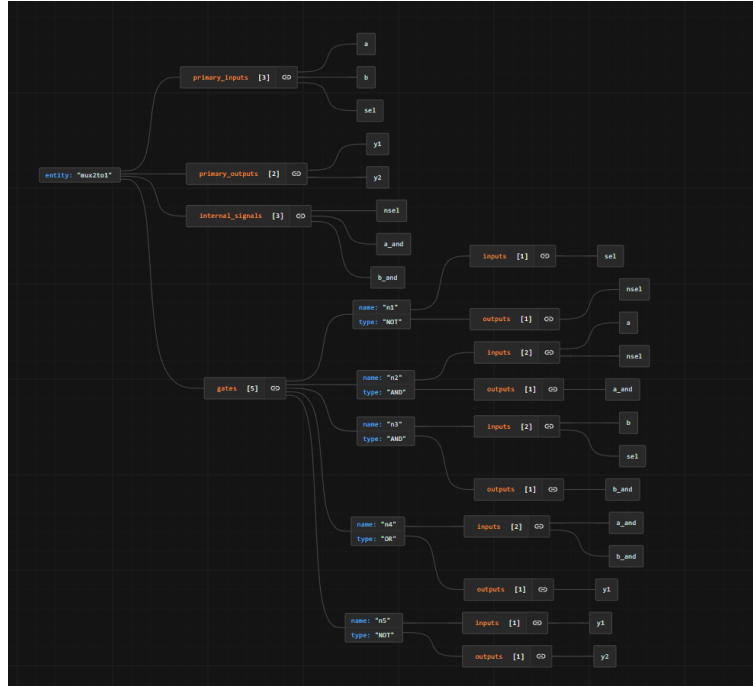


Figure 1: Graph representation of circuit generated from JSON

4 Implementation

4.1 Project Structure

```

A1 LOGIC_SIMULATOR/
├── data/
│   ├── inputs/ ..... Input VHDL + vectors
│   ├── intermediate/ ..... JSON + logic expressions
│   └── outputs/ ..... Simulation results
├── src/
│   ├── config.py ..... Path management
│   ├── parse_gate_level.py ..... Parse VHDL → JSON
│   ├── generate_slc.py ..... JSON → Logic Expressions
│   └── compiled_code_simulation.py ..... Run simulations
├── main.py ..... Entry point
├── README.md ..... Documentation
└── requirements.txt ..... Dependencies

```

4.2 Workflow

1. **Parse Gate-Level Netlist** Reads .vhd1 file, extracts gates and signal connections, stores results as .json.
2. **Generate Logic Expressions** Reads JSON, creates simplified Boolean equations, saves into slc.txt.
3. **Run Simulation** Reads input vectors, evaluates equations with a safe AST-based evaluator, outputs results in outputs.txt.

4.3 Example

Input Netlist (VHDL):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2to1 is
    Port ( a : in STD_LOGIC;
          b : in STD_LOGIC;
          sel : in STD_LOGIC;
          y1 : out STD_LOGIC;
          y2 : out STD_LOGIC);
end mux2to1;

architecture gate_level of mux2to1 is
    signal nsel, a_and, b_and : STD_LOGIC;
begin
    n1: not_gate port map (sel, nsel);
    n2: and_gate port map (a, nsel, a_and);
    n3: and_gate port map (b, sel, b_and);
    n4: or_gate port map (a_and, b_and, y1);
    n5: not_gate port map (y1, y2);
end gate_level;
```

Simulation Input (inputs.txt):

```
a b sel
0 0 0
0 1 0
1 0 1
1 1 1
```

Output (outputs.txt):

```
OUTPUT
0
0
0
1
```

5 Results

- Successfully simulated multiple netlists (multiplexers, adders).
- Verified outputs match expected truth tables.
- Intermediate JSON and SLC files enable debugging and transparency.
- Modular codebase ensures extensibility for future CAD flows.

6 Conclusion

This project demonstrates a complete flow for VHDL gate-level simulation:

Parsing $\rightarrow$ Transformation $\rightarrow$ Simulation $\rightarrow$ Output

It forms the foundation for more advanced CAD tools such as synthesis, optimization, and timing simulation.

Future Improvements

- Extend support for sequential circuits (flip-flops, latches).
- Add GUI for visualization.
- Extend parser to handle more VHDL constructs.